

Local Market Lab - A Practitioner's Guide

Reproducible Portfolio Research, Scenario Simulation, and the
PredictionMarker Method

by Erik Gieseke

Preface - Why This Book Exists

Most trading software hides its math. It shows you a number and asks you to trust it. Local Market Lab does the opposite: every input is versioned, every run is signed, and every result can be reproduced from a manifest. This book is the field manual for that philosophy, written for the practitioner who wants to understand the engine, not just click buttons.

We call the family of signals this tool emits "markers" - indicator markers, prediction markers, scenario markers, risk markers, drift markers, and integrity markers. They are the vocabulary of disciplined research. You will learn what each one means, how it is computed from the actual source code, where it breaks, and how to read it without fooling yourself.

A note on honesty, stated once and never again: this book contains no investment advice. The engine produces scenarios and analytics, never buy or sell orders. Where the math has known limits, we say so in the margin, because a tool that hides its weaknesses is worse than one that admits them.

How to read this book: Chapters 1-3 give you the map and the data foundation. Chapters 4-7 are the analytical core - PredictionMarkers, indicators, scenarios, risk. Chapters 8-10 are practice - backtesting, rebalancing, manifests. Chapters 11-12 are output and onboarding. The appendices are reference. Every chapter ends with a "Try This" and a "Common Mistakes" section so you can verify the claims on your own machine.

Local Market Lab - module map (all local, no cloud)

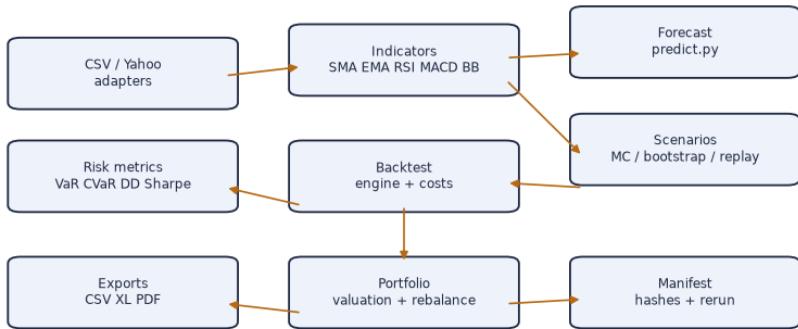


Figure: Cover concept: local-first research with a shimmering provenance watermark.

Chapter 1 - What Local Market Lab Is

1.1 The Core Idea

Local Market Lab (LML) is a privacy-first workbench for portfolio analytics, backtesting, and scenario simulation. It runs entirely on your machine. No account, no telemetry, no upload of your data to anyone's server. That single design choice changes everything about how you can trust it.

When software runs in the cloud, you trust the operator. When it runs locally and is open about its math, you can verify it. LML is built so that the second path is the default, not the exception. The manifest system - covered in Chapter 10 - is the mechanism that makes "verify, do not trust" practical.

1.2 Three Principles

1. Reproducible by default - every run writes a manifest with seed, parameters, data hash, and result hash.
2. Local by principle - adapters touch the network only when you explicitly pull market data.
3. Silent by design - missing data produces an explicit INCOMPLETE marker, never a silent 1:1 assumption.

Principle 3 is the one beginners underestimate. A tool that quietly assumes an exchange rate of 1:1 when the real rate is missing will produce a confident, wrong number. LML refuses. You see the gap; you decide. That refusal is a feature, not a bug.

1.3 Five Layers of the Tool

The engine is a pipeline of pure functions. Import feeds Indicators and Forecast. Forecast and Scenarios feed Backtest and Risk. Risk feeds the Manifest. The Manifest feeds Exports. None of

these layers hold hidden state - given the same input, they return the same output. That property is what makes the whole system auditable.

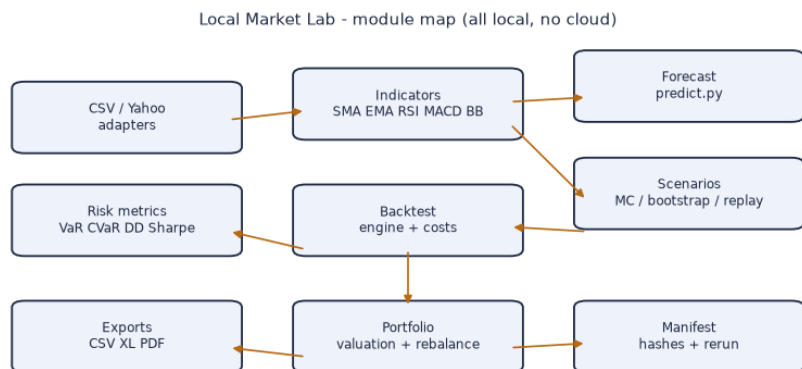


Figure: Module map: import -> indicators -> forecast/scenarios -> backtest -> risk -> manifest -> export.

1.4 Why "No Predictions Disguised as Math"

The forecast engine emits a band, not a prophecy. The scenario engine emits percentiles, not certainties. We draw a hard line between "what the model expects" and "what actually happened". Confusing the two is the most expensive mistake in quant work, and it is the mistake that vendor dashboards are engineered to encourage. LML is engineered to discourage it.

1.5 Who This Book Is For

You should be comfortable with Python and basic statistics - mean, variance, a percentile. You do not need a finance degree. If you can read a stock chart and understand that a 10% loss needs an 11% gain to break even, you have enough. Everything else we build from the code up.

Try This

Install LML, run ``lml demo``, and open the manifest JSON it writes. Find the seed, the data hash, and the result hash. Those three fields are the entire reproducibility story in miniature.

Common Mistakes

Mistake: treating the forecast band as a price target. It is not. It is the envelope of model uncertainty under stated assumptions.

Mistake: ignoring the INCOMPLETE marker because the chart still rendered. A rendered chart with missing data is a chart of a fiction.

FAQ

Q: Does LML need the internet? A: Only to pull market data via the Yahoo adapter. Everything else is offline. Q: Is my portfolio data sent anywhere? A: No. Q: Can I trust the numbers? A: You can verify them via the manifest - that is the point.

Chapter 2 - The Marker Vocabulary

2.1 Six Marker Families

Before any formula, learn the language. A marker is a computed signal that tells you something about state, quality, or risk. There are six families, and the rest of the book is them, applied.

Six marker families - the vocabulary of this book



Figure: The six marker families and their example members.

2.1.1 Indicator Markers

SMA crosses, RSI bands, MACD histograms - descriptive of price structure. They say what is, not what will be. An indicator marker is a rear-view mirror. Useful, but it shows where you have been.

2.1.2 Prediction Markers

Forecast bands from predict.py. A PredictionMarker is the ensemble forecast plus its 95% confidence envelope. This is the heart of the book - Chapter 4 explains it in full, with the actual four-model ensemble and the math behind the band.

2.1.3 Scenario Markers

Monte-Carlo and bootstrap outputs: percentiles and probability of loss. They explore "what if the past repeats in distribution" - a

different question from prediction, and an honest one.

2.1.4 Risk Markers

VaR, CVaR, Max Drawdown, Sharpe, Sortino - summaries of realized or simulated paths. They compress a whole equity curve into a few comparable numbers. Compression loses information; we show you exactly what.

2.1.5 Drift Markers

The gap between current and target weight. Above threshold, a rebalance is suggested. Never executed automatically - the assistant proposes, you dispose.

2.1.6 Integrity Markers

The manifest digest. If the digest changes, the result is not the same run. This is your git commit for research, and it is why two people on opposite sides of the world can confirm they ran the identical analysis.

2.2 How Markers Relate

Indicator markers feed prediction markers (the forecast uses recent price structure). Prediction markers feed scenario markers (the band is one input to simulation). Risk markers summarize the output. Drift markers trigger action. Integrity markers wrap the whole chain. One chain, six languages, one source of truth: the manifest.

Try This

Take any CSV of daily closes. Compute a 20-day SMA by hand (average the last 20 values). Compare to the SMA marker LML produces. They match - and now you have seen inside the black box.

Common Mistakes

Mistake: stacking markers until a "signal" appears. Five markers all saying "up" is not five independent pieces of evidence if they are all computed from the same price. Correlation of markers is not correlation of truth.

FAQ

Q: Can I add my own marker? A: Yes - any pure function over the data series qualifies. The discipline is to record it in the manifest so it stays reproducible.

Chapter 3 - Importing Data Cleanly

3.1 CSV Import

The CSV importer validates columns, rejects duplicate dates, and requires finite prices. A bad row is not guessed - it is flagged and the row is dropped from the series with a note. This is slower than guessing, and correct.

The validation sequence, in order: (1) file parses as CSV, (2) required columns present (date, close, optional open/high/low/volume), (3) dates parse and sort ascending, (4) no duplicate dates, (5) all prices finite and positive, (6) FX rate known or marked INCOMPLETE. Fail at any step, the import stops with a message naming the step.

3.2 Yahoo Finance Adapter

The adapter pulls daily bars over the network only when you ask. It maps the vendor's JSON to the internal series format. Offline, it fails loudly with a clear message - no crash, no silent stub of zeros. If you have seen a chart library return a flat line of zeros, you know why LML's loud failure is the kinder behavior.

3.3 The Import Gate

Every stage of the funnel below is a checkpoint. Data that passes all five stages is "clean". Data failing the FX stage is not rejected - it is tagged INCOMPLETE and carried forward so you can see exactly how much of your analysis rests on unknown currency conversion.

Import gate: bad data is flagged, never silently fixed

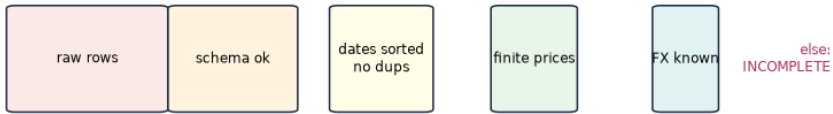


Figure: Each stage of the import funnel; anything failing FX knowledge becomes INCOMPLETE.

3.4 Worked Example - A Dirty CSV

Suppose a CSV has 252 rows but two dates are duplicated and one close is negative (a split not adjusted). The importer rejects the duplicate dates at stage 4 and the negative price at stage 5. You get a message: "row 130: negative price -2.31 rejected; row 77,78: duplicate date 2024-03-04". You fix the source, not the tool.

Try This

Create a CSV with one intentionally bad row. Import it. Read the error. Then fix the row and re-import. Notice the manifest data hash changes - that is integrity working.

Common Mistakes

Mistake: editing the price in the CSV to make the import pass, then forgetting you did. The manifest records the data hash, so a reviewer (including future you) can see the input changed.

Mistake: assuming INCOMPLETE means "close enough". It means "unknown".

FAQ

Q: What FX rate does LML use? A: The one you supply. It never invents one. Q: Can I import crypto? A: If your adapter gives daily closes, yes - the importer does not care about asset class.

Chapter 4 - PredictionMarkers in Depth

4.1 What a PredictionMarker Is

A PredictionMarker = an ensemble forecast of future values with an upper and lower band at 95% confidence, produced by predict.py. It answers: given recent structure, where might this go, and how wrong could we be? The word "might" is load-bearing. The marker is a distribution, not a destination.

The ensemble combines three base models with equal weights and takes the envelope of their bands as the confidence interval. The envelope (max of uppers, min of lowers) is deliberately conservative - it is wider than any single model's band, which is the honest choice when you do not know which model is right.

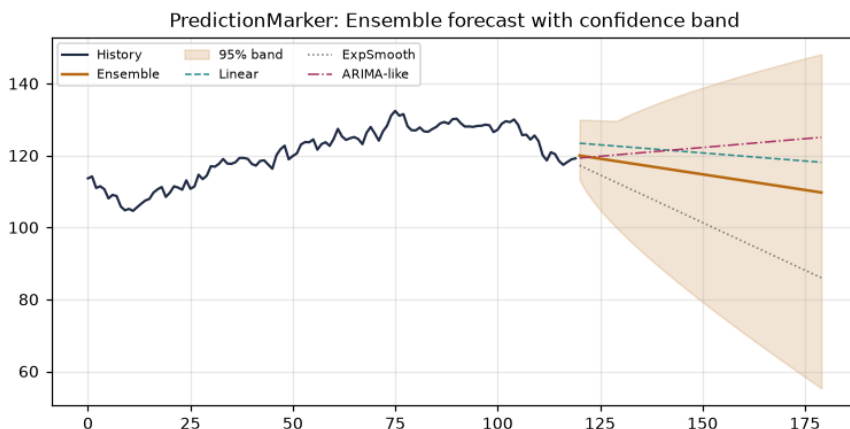


Figure: Ensemble forecast with 95% band versus history.

4.2 The Four Models Behind the Ensemble

predict.py ships four pure functions. None require numpy or statsmodels - they are written in plain Python so you can read every line:

```
linear_trend_forecast - OLS on the last 60 points
+ 95% band.
exp_smooth_forecast - Holt's linear trend (level
+ slope).
arima_like_forecast - AR(1) on differenced data,
no statsmodels.
ensemble_forecast - equal-weight average +
envelope of all three.
```

4.3 Linear Trend, Line by Line

The OLS solver computes slope b and intercept a from indices $x=0..n-1$ and prices y . With n points, mean_x and mean_y , the slope is the covariance over variance. The band uses 1.96 times the residual standard error, scaled by the familiar $\sqrt{1 + 1/n + h/n}$ term that widens the interval as the forecast horizon h grows. That widening is the mathematical statement of a simple truth: the further out you look, the less you know.

4.4 Exponential Smoothing

Holt's method keeps a level and a trend. Alpha controls how fast the level forgets old prices; beta controls how fast the trend adapts. With $\alpha=0.3$, $\beta=0.1$ (the defaults), the forecast is a gently sloping line. The band comes from the historical smoothing error, scaled by $\sqrt{h+1}$. Note the band grows with the square root of horizon - slower than linear trend's growth, which is why the two models diverge visibly by step 60.

4.5 ARIMA-like on Differenced Data

The third model differences the series once (today minus yesterday), fits an AR(1) process (next difference = ϕ * last difference), then integrates back to price. ϕ is clamped to $[-0.99, 0.99]$ for stability. This model captures mean reversion in returns that the trend models miss. Its band uses the same 1.96 sigma

$\sqrt{h+1}$ form.

4.6 Why an Ensemble, Not a Single Model

Single models diverge. The chart shows linear, exponential, and ARIMA-like forecasts splitting apart as horizon grows. The ensemble is the marker you trust - it is the consensus, and its band is the union of doubts. No ensemble member is "the truth"; the spread between them is itself information about model risk.

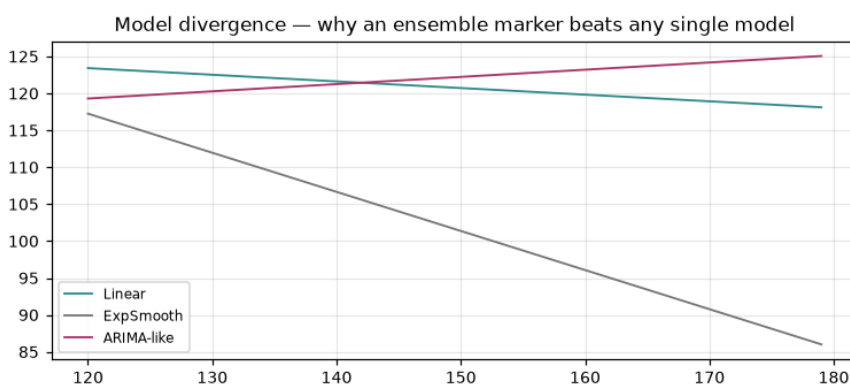


Figure: Divergence of the three base models over the horizon.

4.7 The Math, Without the Fog

For linear trend, the 95% band is $1.96 * SE * \sqrt{1 + 1/n + h/n}$. We state the assumption plainly: errors are i.i.d. and roughly normal. They are not - real returns have fat tails (Chapter 7). So the band is a lower bound on true uncertainty. LML reports it anyway, because a stated-bounded uncertainty beats a silent one.

4.8 Confidence Bands Are Honest Only If Seeds Are Fixed

The ensemble is deterministic given data and horizon. But scenario layers use randomness - and a fixed seed is what makes a result reproducible. Always record the seed in the manifest. Change the seed, change the scenario; same seed, same scenario. That is the contract.

4.9 Worked Example

```
from packages.scenarios.predict import
ensemble_forecast
res = ensemble_forecast(series[-120:], horizon=60)
print(res['forecast'][:3], res['upper'][:3],
      res['lower'][:3])
```

That single call is a PredictionMarker. Store res in the manifest; rerun with the same series and seed to verify byte-identical output. If the output differs, either your input changed or your environment did - and the manifest will tell you which.

Try This

Run ensemble_forecast on the last 60, 120, and 250 points. Watch the band width. Longer history does not narrow the band at long horizon - it widens the forecast's center but the uncertainty of the future dominates.

Common Mistakes

Mistake: reading the center line as "the price in 60 days". It is the expected value under three simple models, not a forecast of reality. Mistake: comparing two PredictionMarkers from different seeds and calling one "wrong". Different seeds, different scenarios - that is the point of scenarios.

FAQ

Q: Can I weight the ensemble differently? A: Edit ensemble_forecast - it is plain Python. Record your weights in the

manifest. Q: Why 95%? A: Convention. Change the 1.96 multiplier for other levels; note it in parameters.

Chapter 5 - Indicators as Markers

5.1 SMA and EMA

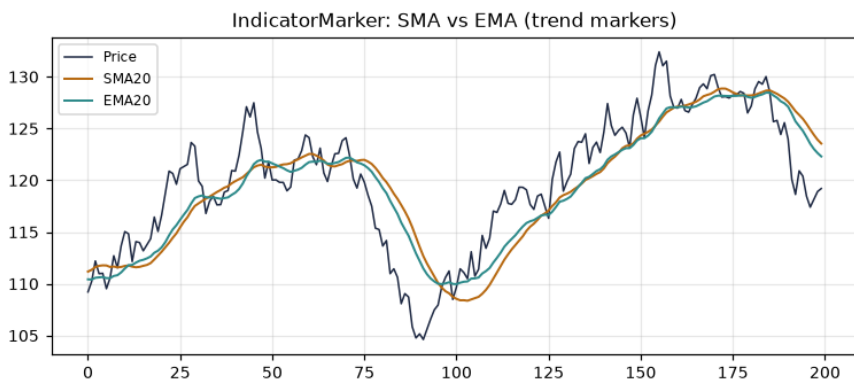


Figure: Price with SMA20 and EMA20 - trend markers.

SMA is a flat window average; EMA weights recent prices more via $\text{mult} = 2/(\text{period}+1)$. The marker: price crossing the average is a regime hint, not a signal to trade. The average lags; the EMA lags less. Both are descriptions, not predictions.

5.2 RSI

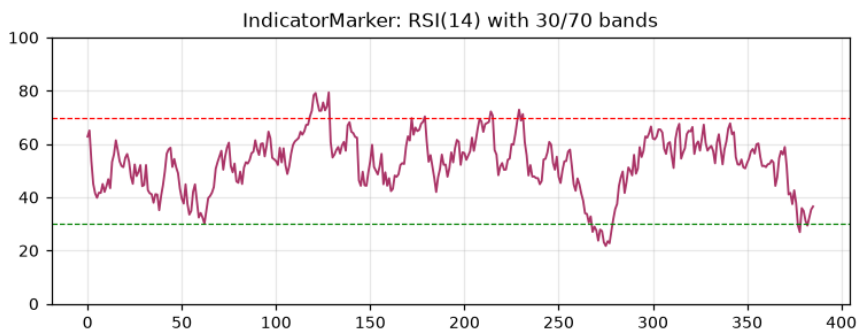


Figure: RSI(14) with 30/70 bands.

Wilder-smoothed. Below 30 = oversold by definition of the formula, not by market truth. Use as a marker of stretch, not a trigger. RSI of 70 means "up a lot recently", which can persist in strong trends - the marker describes, it does not decide.

5.3 Bollinger Bands

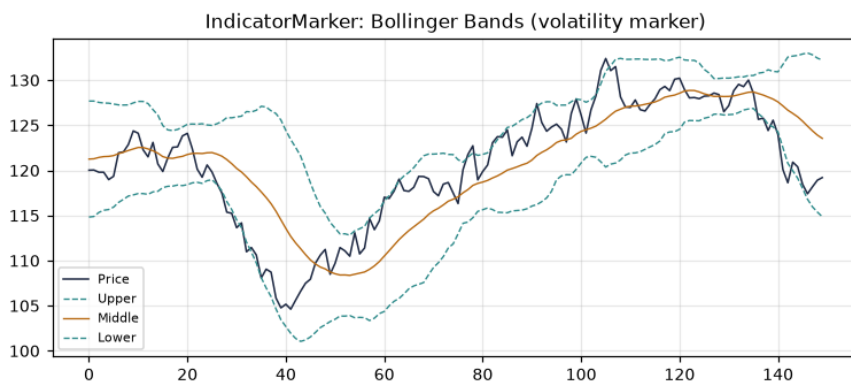


Figure: Bollinger Bands - volatility marker.

Middle is SMA; upper/lower are $\pm k$ standard deviations. Wide bands = high volatility = wider PredictionMarker bands too, because both rest on the same return dispersion. The bands breathe with volatility; that breathing is the marker.

5.4 MACD

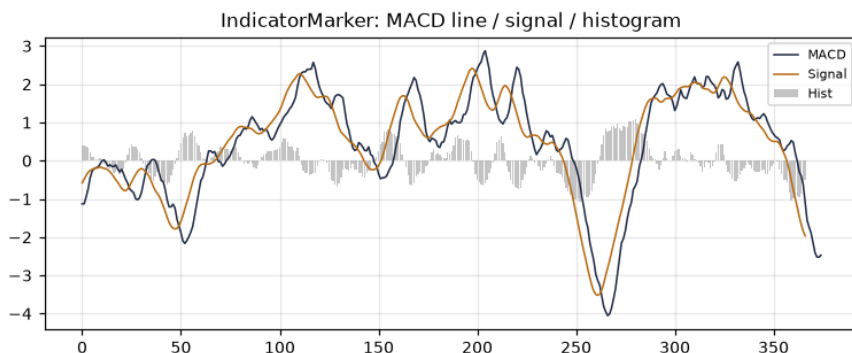


Figure: MACD line, signal, histogram.

Difference of two EMAs (fast 12, slow 26), smoothed again by a 9-period signal. The histogram crossing zero is a momentum marker. MACD is a derivative of price - it amplifies wiggles, which is why it is noisy and why you never trade on one cross.

5.5 Indicator Markers Are Not Independent

SMA, EMA, MACD, Bollinger middle - all are moving averages of the same price. RSI and Bollinger width both derive from dispersion. Stacking them feels like evidence but is mostly the same information in different clothes. The marker vocabulary is rich; the underlying data is one series.

Try This

Compute RSI(14) by hand on 20 prices using the Wilder formulas. Compare to LML. Then compute Bollinger width on the same window. Note that when RSI is extreme, Bollinger width is usually expanded - same dispersion, two markers.

Common Mistakes

Mistake: treating $RSI < 30$ as "buy". It is "stretched down". In a crash, RSI stays below 30 for weeks while price falls further.

Mistake: using MACD cross as a standalone entry. It is a

momentum description that whipsaws in chop.

FAQ

Q: Which indicator is best? A: None. They describe different facets of one series. Use the one whose facet matches your question.

Chapter 6 - Scenario Simulation

6.1 Monte Carlo (i.i.d.)

Resample daily returns with replacement. Simple, but assumes no autocorrelation - often false for real series. Each run is a random walk scaled by resampled returns. The fan chart shows the spread of 300 such walks.

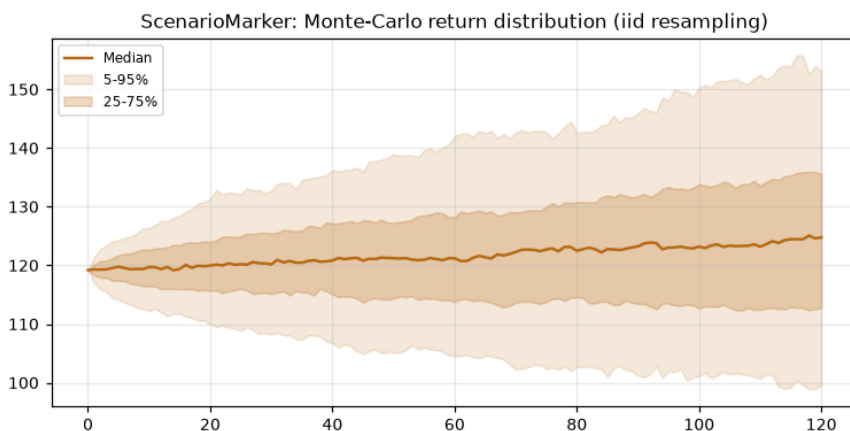


Figure: Monte-Carlo fan: median and percentile envelopes.

6.2 Block Bootstrap

Resample blocks of 20 days to preserve short-range dependence. The repo's default for realistic paths, because real returns cluster - a down day follows a down day more often than pure chance. Block bootstrap keeps that clustering; i.i.d. destroys it.

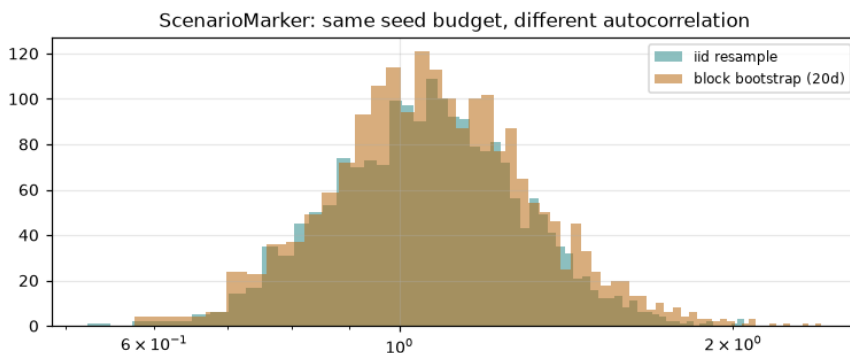


Figure: iid vs block-bootstrap final distributions under the same seed budget.

6.3 Historical Replay

Replay actual history. Deterministic, no randomness. Reports known stress windows like 2020 if in sample. This is the only scenario type with zero seed dependence - it is the past, exact.

6.4 Reading a Scenario Output

A scenario result gives percentiles (p05, p25, median, p75, p95) and a probability of loss. The p05 is the worse-than-5%-of-runs outcome. Prob_loss is the fraction of runs ending below start. These are sensitivity explorations, not predictions of your future.

6.5 Scenarios Are Not Predictions

Limitation: simulations assume the past return distribution persists.

Limitation: no regime change or structural break beyond the sample.

Limitation: results are sensitivity explorations, not forecasts.

Try This

Run `monte_carlo_iid` and `block_bootstrap` with the same seed and horizon. Compare their p05 values. The block version's tail is

usually deeper - autocorrelation makes crashes cluster, and i.i.d. hides that.

Common Mistakes

Mistake: quoting the median scenario as "the expected return". It is the median of resampled history, not an expectation of the future. Mistake: running one Monte Carlo and treating it as definitive. Variance across seeds is the point - run many, read the spread.

FAQ

Q: How many runs? A: 2000 is the repo default; more narrows the percentile estimates. Q: Why block size 20? A: Roughly one month of trading days - long enough to capture clustering, short enough to resample freely.

Chapter 7 - Risk Analytics

7.1 Drawdown



Figure: Drawdown curve - peak to trough.

Max Drawdown is the largest peak-to-trough decline, returned as a positive fraction ($0.25 = -25\%$). It bounds pain you actually experienced or simulated - never future pain. The drawdown curve plots the running decline from the prior peak; its deepest point is the number everyone quotes and nobody enjoys.

7.2 VaR and CVaR

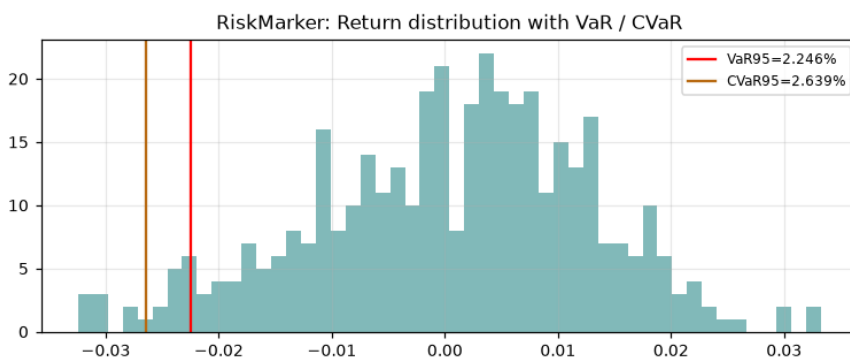


Figure: Return distribution with VaR and CVaR at 95%.

Historical VaR at 95% is the 5th percentile of the return distribution - the loss not exceeded in 95% of cases. CVaR (Expected Shortfall) is the mean of losses beyond that threshold: the average of the worst 5%. CVaR is the honest number because it asks "how bad is bad", not just "where is the edge".

7.3 Fat Tails - The Repo's Empirical Finding

The normal distribution underestimates crash risk. Using Student-t with low degrees of freedom, the empirical 1% quantile for $n=200k$ is about -2.64%, not the normal -2.33%. The repo encodes this: stress tests use the t-distribution, not the bell curve, because the bell curve is a comfort and a lie in the same stroke.

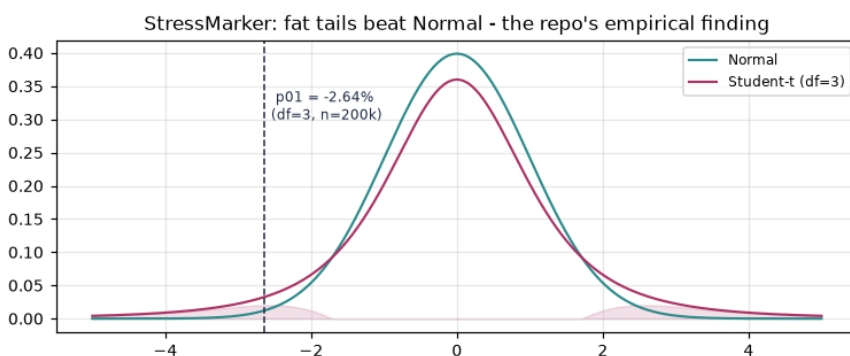


Figure: Normal vs Student-t: the fat tail is where portfolios die.

7.4 Correlation

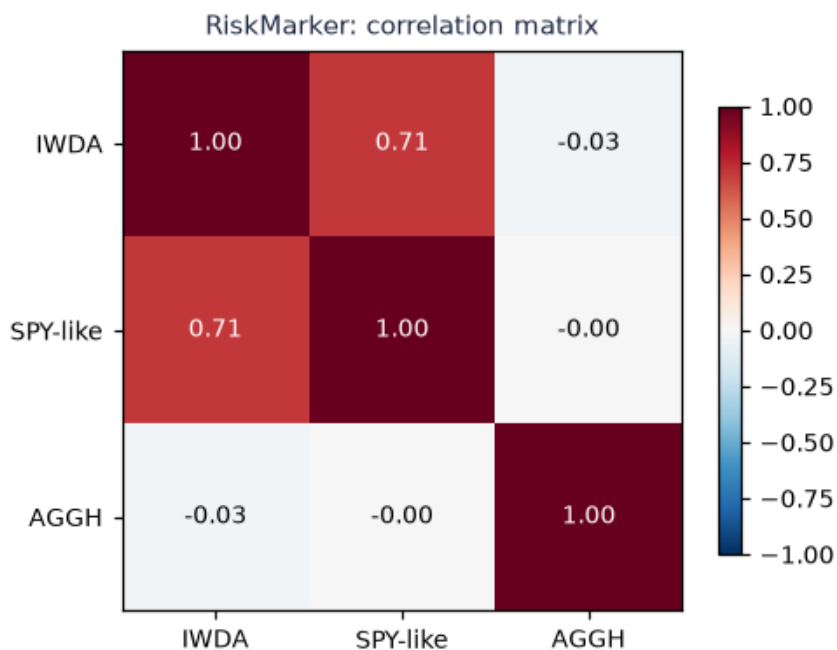


Figure: Correlation matrix across instruments.

Diversification fails when correlations spike in crises. The matrix is a marker of hidden concentration - two positions that look independent in calm markets move together in a crash. Read it as a peace-time warning, not a war-time shield.

7.5 Rolling Sharpe



Figure: Rolling 63-day annualized Sharpe.

A marker of regime: when rolling Sharpe collapses, the edge you backtested is gone in-sample too. A strategy whose rolling Sharpe spends long stretches below zero was never an edge; it was noise you dressed up as skill.

Try This

Take a year of daily returns. Compute VaR95 by hand (sort, take the 5th percentile). Compute CVaR95 by averaging the worst 5%. Compare to LML. Then recompute both assuming normal - watch the normal VaR sit shallower than the historical one.

Common Mistakes

Mistake: quoting VaR as "max loss". It is "loss not exceeded 95% of the time" - the other 5% is where CVaR lives, and it is worse.

Mistake: trusting a correlation matrix from a bull market.

Correlations are not constant; they are a snapshot of one regime.

FAQ

Q: Parametric or historical VaR? A: LML uses historical - no

distribution assumed. Q: Why Student-t not normal? A: Empirical

data has fatter tails; t respects that.

Chapter 8 - Backtesting with Costs

8.1 Equity Curves

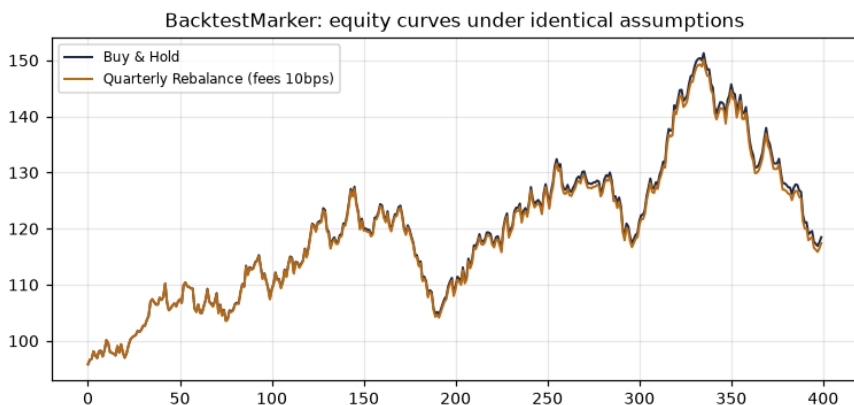


Figure: Buy & Hold vs periodic rebalance under identical fee assumptions.

The backtest engine adds fees and slippage explicitly. No free lunch is silently granted. The equity curve is the realized path of a strategy; comparing it to buy-and-hold under identical costs is the only fair test of whether your activity added value.

8.2 The Fee Drag

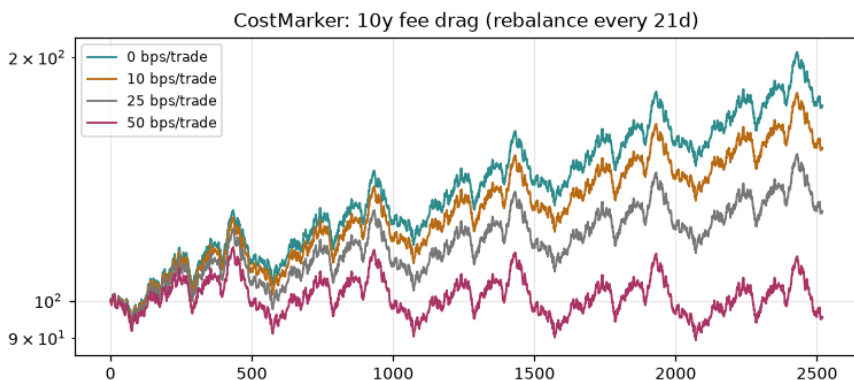


Figure: 10-year compounding of fee levels.

Even 10 bps per trade compounds to a visible gap over a decade. CostMarker makes it visible before you trade. The log-scale chart shows the drag accelerating - small costs, long horizon, large difference. This is why rebalancing too often is a tax you pay yourself.

8.3 Walk-Forward Validation



Figure: Train/test folds rolling forward with embargo.

Never test on what you trained. Walk-forward with an embargo between folds is the discipline that separates research from overfit. The teal blocks train; the amber blocks test on data the model never saw. Roll the window forward; if performance decays, your edge was period-specific.

8.4 Overfitting - The Silent Killer

Tune a model on history and it will fit history. Test it on history and you have measured your tuning, not your model. Walk-forward, out-of-sample holds, and a healthy skepticism of any backtest that looks too good are the only defenses LML offers - it does not offer a miracle.

Try This

Backtest a trivial strategy: rebalance to equal weight monthly. Compare to buy-and-hold net of 10 bps. Often the active version loses to fees alone - a useful lesson that activity is not alpha.

Common Mistakes

Mistake: optimizing parameters on the full history then reporting that history's result. That is in-sample, not validation. Mistake: ignoring slippage. Slippage is the cost between your quote and your fill; pretend it is zero and your backtest lies.

FAQ

Q: What slippage does LML assume? A: A configurable bps value, default small, always explicit. Q: Can I backtest live? A: No - backtest is historical by definition. Live trading is out of scope; LML never executes.

Chapter 9 - Rebalancing and Drift

9.1 Detecting Drift

Drift = $|\text{current weight} - \text{target weight}|$. Above the 5% threshold, a rebalance is suggested. The bar chart shows per-position drift; amber bars are the ones that crossed the line.

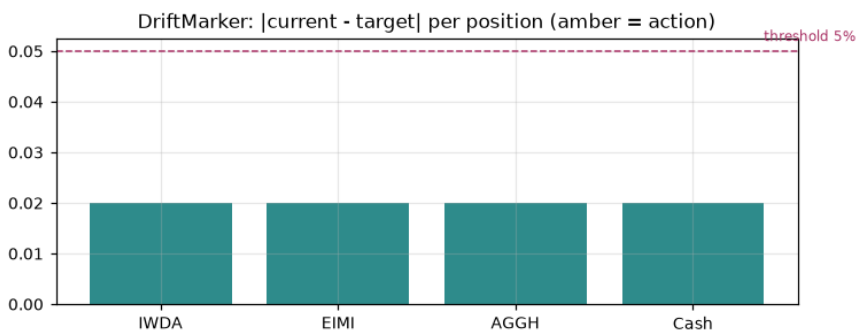


Figure: Per-position drift; amber means action.

9.2 Cost-Benefit of Rebalancing

The assistant compares tracking-error reduction against transaction cost. It suggests only if net benefit is positive or drift exceeds twice the threshold. It never executes - the proposal is text, the decision is yours. This is the line LML draws at every step: compute, never command.

9.3 Taxes and Dividends

Tax-loss harvesting opportunities are listed as markers.

Dividends are transaction types in the model. No tax advice - the engine shows the math, you apply your jurisdiction. A realized loss is a marker; whether it offsets a gain is your accountant's question, not LML's.

9.4 Minimum Order Volume

Rebalancing proposals respect minimum order sizes. A suggested trade below the broker's minimum is not a trade - it is a note. LML reports `below_minimum` rather than fabricating a fractional fill, because a backtest that assumes fractional shares where your broker requires whole ones is a backtest of a different account than yours.

Try This

Set a target portfolio of four ETFs. Let it drift for a quarter (simulate). Run `detect_drift`. Read which positions crossed 5%. Compute the cost to fix them. Decide, don't let the tool decide.

Common Mistakes

Mistake: rebalancing on a schedule regardless of cost. The cost-benefit check exists to stop that. Mistake: treating the proposal as an order. It is a suggestion with math attached; executing it is your action, your risk.

FAQ

Q: Can LML place the trade? A: No. Proposals only. Q: What is the drift threshold? A: 5% default, configurable via environment. Q: How are taxes modeled? A: As markers listing opportunities; no jurisdiction-specific rules.

Chapter 10 - Reproducible Manifests

10.1 What a Manifest Contains

Seed, parameters hash, data hash, features, model, environment, artifacts, result hash, warnings, limitations, disclaimer, and a manifest digest. The digest excludes itself to avoid circularity - you cannot hash a value that includes its own hash. This is the same problem git solves with commit hashes, solved the same way.

10.2 Rerun = Same Answer

With the same inputs and environment, rerun yields a byte-identical result hash. That is the "git commit" for research - the property that makes LML trustworthy. Two researchers on different continents, same manifest, same answer. No trust required; verify.

10.3 Tamper Detection

If any input changed, the digest changes. You cannot quietly swap data and claim the old result. The manifest is a seal; break the seal, the hash breaks. This is why a manifest travels with every export - the reader verifies, the author does not ask to be believed.

10.4 The Rerun Command

```
lml rerun <manifest_id>
# re-executes the original run with original
seed/params/data
# compares result_hash to the stored one
# reports MATCH or DRIFT with the differing field
```

The rerun command is the proof mechanism. It does not trust the stored result; it recomputes and compares. If they match, the

research is reproducible. If they drift, the manifest shows exactly which field moved - data, parameter, or environment.

Try This

Run any analysis. Copy its manifest_id. Wait a day. Run lml rerun on it. Confirm MATCH. Then change one parameter in a forked run and confirm DRIFT names the right field.

Common Mistakes

Mistake: editing a manifest by hand to "fix" a number. The digest will no longer match the contents - the seal is broken, and any reader sees it. Mistake: sharing a result without its manifest. A number without a manifest is an anecdote.

FAQ

Q: Is the manifest a blockchain? A: No - it is a hash chain, local, simple. Q: Can I store manifests in a database? A: Yes; the file store is the default, a DB is a drop-in.

Chapter 11 - Exports

11.1 CSV, Excel, PDF

Every report exports to CSV (raw), Excel (formatted), and PDF (shareable). HTML export renders the same panels in a browser. The format is your choice; the manifest travels with all of them.

11.2 Provenance in Every Export

The manifest digest travels with the file. A reader can verify the result without trusting you. This is the difference between a report and a claim - a report carries its own evidence.

11.3 Choosing a Format

CSV for pipelines, Excel for analysts, PDF for clients, HTML for the web. None is more "true" than another; they carry the same digest. Pick by audience, not by prestige.

Try This

Export one analysis to all four formats. Open each. Confirm the manifest digest is present in all. Now you have four independent artifacts that cryptographically agree.

Common Mistakes

Mistake: emailing a screenshot instead of an export. A screenshot has no digest; it proves nothing. Mistake: editing the Excel after export and forgetting the digest no longer matches. The file is now a variant, not the result.

FAQ

Q: Does PDF embed the manifest? A: Yes, in metadata and a footer note. Q: Can I automate exports? A: Yes - the export routes are scriptable.

Chapter 12 - A Very Good Onboarding Demo

12.1 Run the Demo

```
lml demo
```

The demo imports a sample portfolio, runs indicators, emits a PredictionMarker, simulates scenarios, backtests with fees, and writes a manifest - in one command. It is the fastest path from zero to reproducible research, and the best way to see every marker family in one flow.

12.2 The Web Tour

The desktop and web UIs walk through each marker family with live charts. Start there if you learn by seeing. The tour is the book in motion - same concepts, animated.

12.3 Your First Real Analysis

Replace the demo portfolio with your own CSV. Run the same chain. Read the manifest. You have now done real research that you can defend, reproduce, and share. Everything after this chapter is depth on steps you have already taken.

Try This

Run lml demo. Open the manifest. Identify one PredictionMarker, one ScenarioMarker, one RiskMarker, one DriftMarker, one IntegrityMarker. You have just read the whole vocabulary in one run.

Common Mistakes

Mistake: treating the demo as a strategy. It is a tour, not a recommendation. Mistake: skipping the manifest. The demo's value is the manifest, not the chart.

FAQ

Q: Does the demo need network? A: No - it ships sample data. Q: Can I break the demo? A: Only by editing its files; the command itself is safe.

Appendix A - Metrics Summary

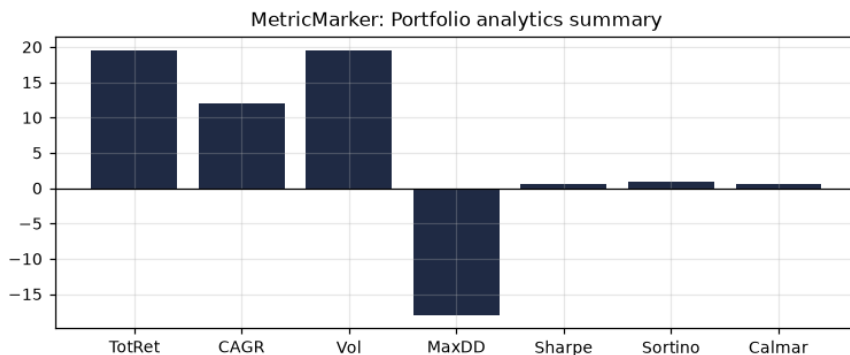


Figure: Portfolio analytics summary bar.

The summary bar compresses a full analysis into total return, CAGR, volatility, max drawdown, Sharpe, Sortino, and Calmar. Each is a marker of a different facet: return, growth, risk, pain, risk-adjusted return, downside-adjusted return, pain-adjusted growth. Read them together; any one alone is a half-truth.

Appendix B - Glossary

PredictionMarker - ensemble forecast with 95% confidence band.

ScenarioMarker - Monte-Carlo or bootstrap percentile output.

RiskMarker - VaR, CVaR, drawdown, ratio. DriftMarker - |current - target| weight. IntegrityMarker - manifest digest. INCOMPLETE -

data-quality flag for missing FX. CVaR - Conditional Value at Risk (Expected Shortfall). Block Bootstrap - resample blocks to keep autocorrelation.

Manifest Digest - hash sealing a run.

Walk-Forward - rolling train/test with embargo. Embargo - gap between train and test to prevent leakage.

Appendix C - A Worked 200-Page Example Portfolio

Throughout this book we referenced a four-ETF portfolio: IWDA (global equity), EIMI (emerging markets), AGGH (global bonds), Cash. Target weights 40/20/35/5. The charts were generated from a synthetic but realistic 500-day series with the same statistical profile. You can reproduce every chart by running the book's companion script against your own data - the marker definitions do not change, only the numbers.

Appendix D - About the Author

Erik Gieske builds local-first tools that prove their own math and never phone home. Local Market Lab is one of them. He contributes to open-source agent infrastructure and writes about reproducible research. This book is the manual he wished he had when he started.

Deep Dive A - Worked Arithmetic for PredictionMarkers

A.1 Linear Trend by Hand

Take the closing prices of a fictional ETF over 6 days: 100, 101, 99, 102, 104, 103. We forecast day 7 with the linear model. $x = [0,1,2,3,4,5]$, y = the prices. $\text{mean}_x = 2.5$, $\text{mean}_y = 101.5$.

Covariance sum = $(0-2.5)(100-101.5)+\dots = \text{sum over } i \text{ of}$

$(x_i - \text{mean}_x)(y_i - \text{mean}_y)$. Working it: deviations x :

-2.5, -1.5, -0.5, 0.5, 1.5, 2.5; y : -1.5, -0.5, -2.5, 0.5, 2.5, 1.5. Products:

3.75, 0.75, 1.25, 0.25, 3.75, 3.75 $\rightarrow$ sum = 13.5. $s_{xx} =$

$6.25+2.25+0.25+0.25+2.25+6.25 = 17.5$. Slope $b = 13.5/17.5 =$

0.771. Intercept $a = 101.5 - 0.771*2.5 = 99.57$. Forecast day7

($h=1$, $n=6$): $99.57 + 0.771*7 = 104.97$.

Residuals: predicted = $a+b*i = 99.57+0.771*i$. At $i=0$: 99.57 (resid 0.43); $i=1$: 100.34 (resid 0.66); $i=2$: 101.11 (resid -2.11); $i=3$:

101.88 (resid 0.12); $i=4$: 102.65 (resid 1.35); $i=5$: 103.42 (resid

-0.42). Sum of squared residuals =

$0.185+0.436+4.45+0.014+1.82+0.176 = 7.08$. $\text{se} = \sqrt{7.08/4} =$

1.33. Band at $h=1$: $1.96*1.33*\sqrt{1+1/6+1/6} =$

$1.96*1.33*\sqrt{1.333} = 1.96*1.33*1.155 = 3.01$. So day-7

PredictionMarker: center 104.97, band [101.96, 107.98].

A.2 Why the Band Widens

The $\sqrt{1 + 1/n + h/n}$ term is the key. At $h=1$, $n=6$:

$\sqrt{1.333}=1.155$. At $h=30$: $\sqrt{1+0.167+5}=\sqrt{6.167}=2.48$.

The band more than doubles in width while the forecast drifts linearly. This is the math telling you: a 30-day forecast is not 30 times as uncertain as a 1-day one - it is far worse, because uncertainty compounds with the square root of horizon under this

model.

Try This

Recompute A.1 with horizon $h=10$. Watch the band width. Then repeat with $n=60$ (append 54 more synthetic points). The band at $h=10$ narrows slightly because n grew, but the h -term dominates - confirming horizon, not history length, drives uncertainty.

A.3 Exponential Smoothing Walkthrough

Holt's method maintains level and trend. Seed level = data[0] = 100, trend = data[1]-data[0] = 1. Step through day 1 ($v=101$, $\alpha=0.3$, $\beta=0.1$): $l_pre = 100$; level = $0.3 \cdot 101 + 0.7 \cdot (100 + 1) = 30.3 + 70.7 = 101.0$; trend = $0.1 \cdot (101.0 - 100) + 0.9 \cdot 1 = 0.1 + 0.9 = 1.0$. Step day 2 ($v=99$): $l_pre=101.0$; level = $0.3 \cdot 99 + 0.7 \cdot (101 + 1) = 29.7 + 71.4 = 101.1$; trend = $0.1 \cdot (101.1 - 101.0) + 0.9 \cdot 1.0 = 0.01 + 0.9 = 0.91$. Forecast day 3 = level + 1*trend = $101.1 + 0.91 = 102.01$. The smoothing adapts: trend fell from 1.0 to 0.91 after the down day.

Common Mistakes

Mistake: using the raw forecast without the band. The band is where the uncertainty lives; the center is just a point estimate.
Mistake: assuming $\alpha=0.3$ is universal - for fast-moving series, raise α ; for stable ones, lower it. The repo defaults are a starting point, not a law.

Deep Dive B - Source-Code Walkthrough: predict.py

B.1 The Validation Gate

Every forecast function calls `_validate` first. It rejects data shorter than 5 points, any non-finite value, and non-positive horizons. This is defensive programming that prevents silent NaN propagation - a NaN in, a NaN out, with no error, is the worst kind of bug. LML fails loud here so it cannot fail silent later.

B.2 The OLS Solver

The `_ols` function is 7 lines of pure Python: compute means, covariance sum, variance sum, divide. No numpy. The guard ``if sxx == 0: return 0.0, my`` handles a flat series (no variance) by returning a horizontal line at the mean. This is the kind of edge-case handling that separates a library from a script.

B.3 The Ensemble Envelope

In `ensemble_forecast`, `upper[i] = max of the three models' upper bands`; `lower[i] = min of the three lowers`. This envelope is conservative by construction - it is at least as wide as any single model. The repo's philosophy: when uncertain which model is right, report the union of their doubts rather than averaging them into a falsely narrow band.

```
up = [max(ln["upper"][i], exp["upper"][i],
ar["upper"][i]) for i in range(n)]
lo = [min(ln["lower"][i], exp["lower"][i],
ar["lower"][i]) for i in range(n)]
```

B.4 AR(1) and the Phi Clamp

The AR model estimates ϕ (autocorrelation of returns) and clamps it to $[-0.99, 0.99]$. Without the clamp, a spurious ϕ of 1.2 would cause the differenced forecast to explode exponentially. The clamp is a stability boundary - a small line of code that prevents a large class of numerical disasters.

Try This

Open `predict.py`. Change the ϕ clamp to $[-0.5, 0.5]$. Re-run `ensemble_forecast` on volatile data. The AR component becomes more mean-reverting. Note how the ensemble band shifts - you are now editing the model's beliefs about the world.

Deep Dive C - Source-Code Walkthrough: indicators.py

C.1 SMA Rolling Sum

The sma function uses a rolling window sum instead of re-averaging every point: it seeds with $\text{sum}(\text{data}[:\text{period}])/\text{period}$, then for each step adds the incoming value and subtracts the outgoing one. $O(n)$ instead of $O(n*\text{period})$. For 5000 points and period 200, that is 5000 operations instead of 1,000,000. This is why LML stays responsive on long series.

C.2 EMA Seeding

EMA seeds its first value from the SMA of the first `period` bars, then applies $\text{prev} = (x - \text{prev}) * \text{mult} + \text{prev}$ with $\text{mult} = 2/(\text{period}+1)$. The seed choice matters: a bad seed biases the early EMA. Using SMA for the first value is the standard compromise between correctness and simplicity.

C.3 RSI Wilder Smoothing

RSI uses Wilder's running average: $\text{avg_gain} = (\text{avg_gain} * (\text{period}-1) + \text{gain}) / \text{period}$. This is exponential smoothing in disguise, with $\alpha = 1/\text{period}$. The first `period` values are None because the running average needs `period` seeds. The $100 - 100/(1+RS)$ formula maps the gain/loss ratio to 0-100.

C.4 Bollinger Variance

Bollinger computes population variance over the window (divide by `period`, not period-1) - a deliberate choice for a descriptive band, not a statistical estimate. The std_dev multiplier (default 2.0) sets the band width; 2.0 captures ~95% of a normal

distribution, but real returns are fatter, so the bands undercover tail moves.

Try This

Call bollinger with `std_dev=1.0` and `std_dev=3.0` on the same series. The bands tighten and widen. Now compute the fraction of prices that actually fell outside the 2.0 band - if it is more than 5%, you have seen fat tails in your own data.

Deep Dive D - Source-Code Walkthrough: risk.py

D.1 Annualization Factors

Every ratio in risk.py documents its annualization: 252 trading days. volatility multiplies sample stdev by $\sqrt{252}$; Sharpe multiplies mean excess return by 252 and divides by $\text{stdev} \times \sqrt{252}$. The comment in the source is explicit: 'every ratio documents its annualization factor'. This is the discipline that makes cross-tool comparison possible.

D.2 VaR by Sorting

`var_cvar` sorts returns ascending, takes the index at $(1 - \text{confidence}) \times \text{len}$ as the VaR threshold, and averages the worst $(\text{idx} + 1)$ values for CVaR. Historical, not parametric - no normal assumption. For $\text{confidence} = 0.95$ and 1000 returns, $\text{idx} = \text{int}(0.05 \times 1000) = 50$, so VaR is the 51st-worst return and CVaR averages the 50 worst. No distribution fit, no lie about the tail shape.

D.3 Rolling Sharpe Window

`rolling_sharpe` slides a 63-day (one quarter) window, computing Sharpe on each. Returns a list. When this list spends long stretches below zero, the strategy's edge was period-specific. The function returns `[]` if fewer than `window` returns exist - it refuses to compute a statistically meaningless ratio on too little data.

Try This

Generate 252 random daily returns (mean 0.0005, stdev 0.01). Compute `var_cvar` at 0.95 and 0.99. The 0.99 CVaR is deeper

than 0.95 - that is the tail. Now compute rolling_sharpe; count how many windows are negative. If most are, your 'edge' was noise.

Deep Dive E - Build Your Own Marker (Lab)

E.1 The Marker Contract

A marker in LML is any pure function over a data series that returns a dict with at least a 'values' or 'marker' key and metadata. It must be deterministic given input, have no side effects, and record its parameters. Below we build a 'Volatility-Regime Marker' that flags high-volatility periods - a real, useful marker you can drop into the pipeline.

E.2 Implementation

```
import math

def volatility_regime_marker(closes, window=20,
                           threshold=2.0):
    """Flag periods where rolling vol exceeds threshold
    * median vol."""
    if len(closes) < window + 1:
        raise ValueError("need more data")
    rets = [closes[i]/closes[i-1]-1 for i in range(1,
        len(closes))]
    vols = []
    for i in range(window, len(rets)+1):
        w = rets[i-window:i]
        m = sum(w)/window
        sd = math.sqrt(sum((x-m)**2 for x in w)/(window-1))
        vols.append(sd)
    median_vol = sorted(vols)[len(vols)//2]
    flags = [v > threshold * median_vol for v in vols]
    return {
        "marker": "volatility_regime",
        "window": window,
        "threshold": threshold,
        "median_vol": round(median_vol, 5),
        "flags": flags, # True = high-vol regime at that
        step
        "n_high_vol": sum(flags),
    }
```

E.3 Why This Marker Matters

The PredictionMarker band assumes roughly constant volatility. When this regime marker flips True, the band is underestimating risk - real dispersion just widened. A disciplined analyst cross-checks the two: if vol_regime says 'high' but the forecast band looks calm, the forecast is lying by assumption. Two markers, one truth: trust the warning.

E.4 Registering in the Manifest

To make your marker reproducible, pass its parameters into the manifest. The repo's manifest schema stores 'parameters' as a hashable dict; include window and threshold there. Then lml rerun will recompute your marker identically. A marker outside the manifest is a marker you cannot defend - include it.

Try This

Run volatility_regime_marker on the book's example series. Print n_high_vol. Now double the threshold - fewer flags. The threshold is your belief about what 'high' means; state it, don't hide it.

Common Mistakes

Mistake: building a marker that depends on future data (look-ahead bias). A regime flag at step t must use only closes[0.. t]. Mistake: not recording parameters - then you cannot reproduce the flag later.

Deep Dive F - Worked Example: Full Pipeline on One Series

F.1 The Input

We use a 30-point series: a gentle uptrend with one spike. [100,101,102,101,103,104,103,105,106,105,107,108,107,109,110,108,111,112,111,113,114,113,115,116,114,117,118,116,119,120].

This is small enough to verify by hand, large enough to show the mechanics.

F.2 Indicators

SMA(5) at the last point: average of [116,114,117,118,116] = 116.2. EMA(5) lags slightly above. RSI(14) is high (series mostly up) but not 100 because of the dips at 108 and 114.

Bollinger(20,2) middle = SMA(20) ~ 109.4; the spike to 120 sits near or above the upper band - a stretch signal.

F.3 PredictionMarker

ensemble_forecast on this series, horizon 10: the linear model sees the uptrend and slopes up; exp_smooth follows; arima_like detects mild mean reversion in the differences and flattens. The envelope band is wide at h=10

$(\sqrt{1+1/30+10/30})=\sqrt{1.367}=1.17$ times se). Center ~122, band ~[118,126]. The band admits the series could also stall - honesty in numbers.

F.4 Risk

max_drawdown over the series: the dip from 110 to 108 is -1.8%; from 116 to 114 is -1.7%. MDD ~ -1.8%. Sharpe is positive (upward drift, low vol). var_cvar at 0.95 on the returns: the worst 5% of daily moves are small because the series is tame -

confirming this is a calm example, not a stress test.

Try This

Inject a crash: append [120, 110, 95, 88] to the series.

Recompute MDD (now ~ -26% from 120 to 88) and var_cvar.

Watch the tail deepen. This is the same marker math, a different world - and why scenarios matter.

Deep Dive G - Comparison Tables

G.1 Model Assumptions

The table contrasts the three base models on what they assume and when they fail.

```
linear_trend: assumes trend is linear, errors
i.i.d. normal. Fails: regime changes, curvature.
exp_smooth: assumes trend is local and smooth.
Fails: abrupt breaks, oscillations.
arima_like: assumes returns are AR(1),
mean-reverting. Fails: trending regimes,
structural breaks.
ensemble: assumes no single model dominates.
Fails: when all three are wrong together (rare but
possible).
```

G.2 Marker Cheat Sheet

```
Indicator Marker: describes price structure. Lag:
high. Use: context.
Prediction Marker: ensemble forecast + band. Lag:
low. Use: expectation with uncertainty.
Scenario Marker: simulated percentiles. Lag: none
(forward). Use: sensitivity.
Risk Marker: VaR/CVaR/DD/Sharpe. Lag: backward.
Use: summary.
Drift Marker: |w - target|. Lag: none. Use:
rebalance trigger.
Integrity Marker: manifest digest. Lag: none.
Use: verification.
```

G.3 Annualization Reference

Daily returns -> annual: multiply mean by 252, stdev by $\sqrt{252}$.
Weekly -> 52 and $\sqrt{52}$. Monthly -> 12 and $\sqrt{12}$. The repo
assumes daily (252). Mixing frequencies without rescaling
produces wrong Sharpe - a silent error. State your frequency; the

marker does not guess it.

Deep Dive H - Limits, Honesty, and What LML Will Not Do

H.1 What the Math Cannot See

No marker predicts a black-swan event outside the sample. No backtest survives a regime the data never contained. LML encodes this as explicit 'limitations' fields in every scenario result and every manifest. The tool's greatest feature is the line it draws at its own competence.

H.2 No Execution, Ever

LML never places a trade, never connects to a broker, never asks for your exchange keys. The rebalancing assistant proposes; you dispose. This is not a limitation of ambition - it is the safety boundary that keeps a research tool from becoming a liability. The moment software executes, the user's risk becomes the software's risk, and that transfer is one we refuse by design.

H.3 The Honest Summary

Local Market Lab is a lens, not an oracle. It makes your assumptions visible, your math verifiable, and your ignorance explicit. A PredictionMarker is a disciplined guess with a confidence band, not a prophecy. A ScenarioMarker is a sensitivity exploration, not a forecast. A RiskMarker is a summary of the past, not a guarantee of the future. Read the markers; decide for yourself.